

Threat Model on Google ADK Agents: An OWASP Agentic Security Initiative Perspective

Clark Jason Ngo
School of Business and Management
City University of Seattle
ngoclarkjason@cityuniversity.edu

Sam Chung
School of Technology & Computing
City University of Seattle
chungsam@cityu.edu

Abstract

Agentic AI based on LLMs and generative AI marks a new frontier in autonomous capabilities but also exposes new security challenges. Google ADK, an open-source framework, facilitates the creation and deployment of intelligent agents but simultaneously expands the attack surface. This paper introduces a threat model tailored for ADK agents, using two foundational examples, and aligns these risks with the taxonomy of the OWASP ASI. Identified threats include memory poisoning, tool misuse, privilege compromise, intent manipulation, cascading hallucinations, and remote code execution risks. We propose corresponding mitigation strategies to strengthen ADK-based AI agents and ensure secure deployment of agentic systems.

Keywords: Agentic AI, AI Agent Systems, Google Agent Development Kit, 4+1 Architectural Views, Threat Modeling, OWASP Agentic Security Initiative,

Problem Statement

The rapid development and widespread adoption of agentic Artificial Intelligence (AI) systems, powered by advanced large language models (LLMs), deliver powerful reasoning and autonomy but introduce unique vulnerabilities. While Google Agent Development Kit (ADK) provides modular and code-first tools for building these agents, its integration of memory, reasoning, and tool use creates a new landscape of threats (Google, 2025). The LlmAgent, which powers ADK examples such as the “greeting_agent” in the first AI agent example called HOS01A and “tool_agent” variations in the second AI agent example called HOS02A, demonstrates the risks stemming from autonomous reasoning combined with external tool invocation (Clark & Sam, 2024a, 2024b; Hancock, 2025).

Traditional methodologies such as Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of Privilege (STRIDE) or Process for Attack Simulation and Threat Analysis (PASTA), which are two of the most well-known threat modeling methodologies, while robust for conventional systems, are limited in their ability to assess AI-specific risks that emerge from dynamic instruction following, contextual memory, and emergent behavior. The Open Worldwide Application Security Project (OWASP) Agentic Security Initiative (ASI) acknowledges this gap by providing a taxonomy of agentic threats (OWASP Foundation, 2024, 2025a, 2025b); however, translating these abstract risks into practical ADK-specific insights remains necessary. Addressing this need requires: 1) Systematic identification and mapping of threats in actual ADK implementations. 2) Translation of OWASP ASI’s agentic taxonomy into actionable safeguards for ADK developers. 3) Proposal of practical mitigations applicable to ADK’s architecture to prevent security blind spots from emergent AI capabilities. In the absence of such tailored threat modeling,

ADK developers may overlook vulnerabilities that can enable privilege escalations, data exfiltration, or systemic manipulation.

Related Work

This paper builds on three primary bodies of knowledge. First, the OWASP Agentic Security Initiative establishes a comprehensive taxonomy that defines emerging risks specific to reasoning, planning, memory, and multi-agent communication in LLM systems. These categories align closely with ADK’s agent behaviors and provide a foundation for systematic risk identification. Second, the OWASP Top 10 for LLM Applications provides broader generative AI vulnerabilities such as prompt injection, data poisoning, insecure outputs, and misinformation, which manifest with heightened severity in agentic systems. Third, Google ADK itself provides flexible agent architectures (LlmAgent, Workflow Agents, and Custom Agents), integration with Gemini models, and external tool use—capabilities demonstrated in the two AI agent examples (HOS01A and HOS02A) (Clark & Sam, 202a, 2025b; Hancock, 2025). These concrete examples expose both opportunities for extended utility and new risks in real-world usage.

Approach

Our methodology combines (1) description of ADK system components, (2) developing ADK examples consists of source code and 4+1 architectural views (Kruchten, 1995), (3) mapping of threats from ASI’s taxonomy to ADK examples, and (4) mitigation strategies tailored for these scenarios.

System Description: ADK agents center on LLMs such as Gemini for reasoning and interaction. Their design includes short-term context maintenance, instruction-driven behavior, and tool invocation. HOS01A presents the simplest form of an agent that greets users, while HOS02A expands capability with built-in and custom tools such as “google_search,” “get_weather,” and “get_current_time,” along with multimodal features such as voice input. All examples are deployed in virtual environments, call Gemini APIs with a GOOGLE_API_KEY, and interact through a local development web app. This architecture highlights both the power of tool-enabled reasoning and the dependence on sensitive keys, APIs, and inputs.

Threat Identification: By aligning ADK functionalities with OWASP ASI, we identify threats across several domains.

- Reasoning vulnerabilities include goal manipulation through prompt injection, deceptive agent behaviors, and insufficient logging leading to repudiation challenges.
- Memory vulnerabilities include poisoning of conversational state and cascading hallucinations, amplifying incorrect outputs.
- Tool-related risks include deliberate misuse of tools, privilege compromise from over-permissive scopes, resource overload through denial-of-service patterns, and the potential for remote code execution if code generation is misapplied.
- Authentication and identity risks revolve around key theft and impersonation.
- Human-centric risks include overwhelming human-in-the-loop reviewers with excessive agent queries or using manipulation tactics to exploit user trust.
- Multi-agent system threats, though not directly demonstrated in HOS01A/02A, are inherent to ADK’s architecture and include poisoned communication, rogue agents, and inter-agent exploitation.

Mitigation Strategies: Secure deployment of ADK agents requires layered defenses aligned with ASI’s playbooks. Restricting tool access and using goal consistency validation help reduce reasoning manipulation. Memory poisoning can be prevented through validation, session isolation, anomaly detection, and verification against trusted sources. Tool misuse is mitigated by sandboxed execution, granular policies, and rate-limiting, while privilege abuse is constrained by strict Role-Based Access Control (RBAC), cryptographic identity, and dynamic time-bound access controls. Authentication risks demand secure key management and behavioral monitoring, while human-in-the-loop threats can be alleviated by workload balancing, trust scoring, and oversight prioritization. For multi-agent environments, encrypted communication, trust scoring, and rogue agent isolation are critical.

Conclusions

This paper develops a threat model specifically for Google ADK agents, grounded in HOS01A and HOS02A implementations and framed by the OWASP ASI taxonomy. We identify threats spanning reasoning manipulation, memory poisoning, tool abuse, privilege compromise, cascading hallucinations, and identity spoofing, while acknowledging additional risks in multi-agent designs. Mitigation strategies extend beyond general security practices, emphasizing tailored countermeasures such as immutable logging, sandboxed execution, anomaly detection, and consensus-based trust validation.

The contribution of this paper is twofold: first, it demonstrates how ASI’s high-level taxonomy applies to concrete ADK examples, and second, it provides actionable guidance for developers deploying agentic applications on Google’s framework. Future work will focus on creating intentionally vulnerable ADK agents to empirically demonstrate exploit scenarios, evaluating deployment of these mitigations in production environments, and extending the threat model to complex multi-agent deployments involving the A2A protocol and emerging features like the Model Context Protocol. Ultimately, ensuring secure adoption of ADK agents is not only a technical task but a prerequisite for building resilient, trustworthy agentic AI ecosystems.

References

- Kruchten, P. (1995). Architectural blueprints—The "4+1" view model of software architecture. *IEEE Software*, 12(6), 42–50. <https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>
- OWASP Foundation. (2024, December 15). *Agentic security initiative: Securing agentic applications*. OWASP GenAI Security Project. <https://genai.owasp.org/2024/12/15/announcing-the-owasp-llm-and-gen-ai-security-project-initiative-for-securing-agentic-applications/>
- OWASP Foundation. (2025a). *Agentic AI – Threats and mitigations* (Agentic Security Initiative Whitepaper). <https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/>
- OWASP Foundation. (2025b). *OWASP Top 10 for large language model (LLM) applications* (Version 1.1). <https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>
- Google, Inc. (2025). *Agent development kit (ADK): Documentation*. <https://google.github.io/adk-docs/>
- Clark, J., & Sam, H. (2025a). HOS01A: Hands-on skills—Greeting agent example. In *ADK samples repository*. Google. <https://github.com/clarkngo/agent-development-kit>
- Clark, J., & Sam, H. (2025b). HOS02A: Hands-on skills—Tool-enhanced agent examples. In *ADK samples repository*. Google. <https://github.com/clarkngo/agent-development-kit>
- Hancock, B. (2025). *Build AI agents & automate workflows (Beginner to Pro)* [Video]. YouTube. <https://youtu.be/P4VFL9nlalA>